

简单根号算法

Part II

wYYSZL

2026.8.11

本节课内容

① 莫队

② 操作分块/根号重构

③ 杂题

目录

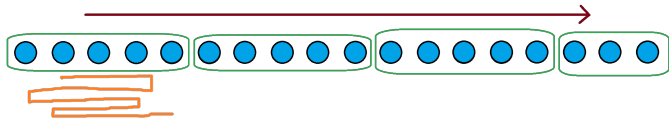
① 莫队

② 操作分块/根号重构

③ 杂题

莫队

莫队是一个相当简单有效的离线做法，支持修改比较麻烦。



图：莫队

我们经常会遇到询问区间 $[l, r]$ 答案的问题。莫队的核心思想十分简单：我们现在已经知道 $[l, r]$ 的答案，我们希望每次通过暴力加入/删除点来扩展/缩小这个区间，得到下一个询问的答案。并按照一定顺序处理询问，使得时间复杂度最小。

下面是一些导出这种顺序的思路历程，初学可以跳过直接看后面的蓝框：

为了让指针移动距离尽可能少，我们可以将询问以某个端点为关键字排序。尽管该端点的移动距离均摊线性，但另一个端点的移动距离无法保证。

注意到两个端点的移动距离分别是 $\mathcal{O}(n)$ 和 $\mathcal{O}(nq)$ 。为了平衡复杂度，自然想到使用 **根号平衡** 的思想。如果将左右端点的移动距离都控制在 $n\sqrt{n}$ 以内，我们就得到了一个时间复杂度为 $\mathcal{O}(n\sqrt{n} \times k)$ 的优秀算法，其中 k 是指针移动的复杂度。

考虑如何分块。设块大小为 B 。我们将所有询问离线下来，并按照左端点 **块编号** 为第一关键字，右端点为第二关键字排序，按照该顺序处理所有询问的时间复杂度为 $\mathcal{O}\left(\frac{nq}{B} + qB\right)$ 。这是因为每个块内右端点的总移动距离不超过 n ，每次询问左端点的移动距离不超过 B 。假设 n, q 同级，取 $B = \sqrt{n}$ ，时间复杂度为 $\mathcal{O}(n\sqrt{n} \times k)$ 。

于是我们有了莫队算法：

莫队算法

将所有询问离线下来，并按照左端点所在块编号为第一关键字，右端点的编号为第二关键字排序。之后，暴力移动指针，从上一个询问到下一个询问。

我们分析复杂度。设块长为 $\sqrt{n}$ ，由于左端点不改变所在块时，右端点只会从左到右扫一次，所以右端点总的移动次数为 $\mathcal{O}(n\sqrt{n})$ 。左端点每次在块内移动或只移动一个块，不超过 $\mathcal{O}(\sqrt{n})$ ，总次数也是 $\mathcal{O}(n\sqrt{n})$ 。

设移动一次指针的代价为 k ，总的复杂度为 $\mathcal{O}(n\sqrt{n} \times k)$ 。

例题 G

G - 小 Z 的袜子

给定长度为 n 的整数序列 $a_1, \dots, a_n$, 以及 m 个询问, 每个询问给出区间 $[l, r]$ 。

从区间 $[l, r]$ 中等概率地随机选取两个不同位置 $i \neq j$, 求 $a_i = a_j$ 的概率。

$n, m \leq 5 \times 10^4$, $a_i \leq n$ 。

例题 G 题解

莫队的关键在于如何加入/删除一个节点，即，如果知道 $[l, r]$ 的答案，如何快速知道 $[l, r+1]$ 的答案，或者 $[l, r-1]$ 的答案。（或者是维护可以快速知道答案的信息，例如后面的例题 I。）允许一些辅助的东西。

针对这道题目，显然这道题需要求得 $a_i = a_j$ 的方案数，然后除以总的方案数即为答案。总的方案数很好求，关键是前一个。

先考虑如何加入节点，加入一个节点 a_{r+1} 时， $a_i = a_j$ 的方案数会加上现在 $[l, r]$ 这个区间内，与 a_{r+1} 相同的 a 的个数。所以我们需要维护每个数出现的个数。这也不难，每次加入节点时对应位置 $+1$ 即可。

这样删去节点也是类似的。

这样，每次移动端点是 $\mathcal{O}(1)$ 的，总的复杂度 $\mathcal{O}(n\sqrt{n})$ 。

莫队 - 实现

莫队在移动端点时的关键过程是加入/删除节点，且会复用。所以一个写法的建议是为加入/删除分别单独写函数，然后每次简洁调用即可。

```
1 void del(int pos);// 传入要删除的点编号
2 void ins(int pos);// 传入要加入的点编号
3 // l++ 和 ++l
4 while (l < query[i].l) del(l++);
5 while (l > query[i].l) ins(--l);
6 while (r < query[i].r) ins(++r);
7 while (r > query[i].r) del(r--);
```

图: 实现

不过也要注意有些题目是不能这样的。

例题 H

H - 异或序列

给定长度为 n 的整数序列 $a_1, \dots, a_n$, 常数 k 。 m 次询问, 每次给出 $[l, r]$, 求满足 $l \leq x \leq y \leq r$ 且

$$a_x \oplus a_{x+1} \oplus \dots \oplus a_y = k$$

的整数对 (x, y) 的数量。

数据规模: $n, m \leq 10^5$, $a_i, k \leq 10^5$ 。

例题 H

H - 异或序列

给定长度为 n 的整数序列 $a_1, \dots, a_n$, 常数 k 。 m 次询问, 每次给出 $[l, r]$, 求满足 $l \leq x \leq y \leq r$ 且

$$a_x \oplus a_{x+1} \oplus \dots \oplus a_y = k$$

的整数对 (x, y) 的数量。

数据规模: $n, m \leq 10^5$, $a_i, k \leq 10^5$ 。

类似上题。注意到 $a \oplus b = k$ 等价于 $a \oplus k = b$, 加入/删除一个点时, 统计窗口内与当前值异或和为 k 的值的出现次数。

例题 I

I - 作业

给定长度为 n 的序列， m 次询问给定 l, r, a, b 。求区间 $[l, r]$ 中介于 a, b 之间的数的个数，和介于 a, b 之间的，在区间内出现过的数值的个数。

说人话：求下面两个值：

- $ans_1 = |\{i \in [l, r] : a \leq a_i \leq b\}|$
- $ans_2 = |\{x \in [a, b] : \exists i \in [l, r], a_i = x\}|$

$$n, m \leq 10^5$$

例题 I 题解

莫队，考虑如何加入/删除以方便求得答案。可以维护两个值域树状数组/线段树（就是下标是值的树状数组），一个正常维护出现次数，一个只有这个位置为 0 时才 +1。

每次询问就直接区间求和即可。总的时间复杂度 $\mathcal{O}(n\sqrt{n}\log n)$ 。

例题 I 题解

莫队，考虑如何加入/删除以方便求得答案。可以维护两个值域树状数组/线段树（就是下标是值的树状数组），一个正常维护出现次数，一个只有这个位置为 0 时才 +1。

每次询问就直接区间求和即可。总的时间复杂度 $\mathcal{O}(n\sqrt{n}\log n)$ 。

但其实仍有优化空间，注意到我们会移动 $\mathcal{O}((n+m)\sqrt{n})$ 次，只会查询 m 次，但是两种操作每次都是 $\log$ 的复杂度，这不平衡。

在朴素分块思想的一开始，我们说分块的一个作用在于平衡，当我们看到不平衡，应该想到分块。让查询复杂度为移动复杂度让位。

例题 I 题解

具体地，我们考虑值域分块。对值域进行分块，每个块维护这 $\sqrt{limit}$ 个值的区间和。每次扩展时，单点修改，只需要修改这个位置的值和区间和即可， $\mathcal{O}(1)$ 。查询时散块 + 整块， $\mathcal{O}(\sqrt{limit})$ 。

离散化后，总的时间复杂度 $\mathcal{O}(n\sqrt{n})$ 。

当然

也可以利用我们昨天讲到的数颜色技巧，即转化为 $lst_i < l$ 的问题，变成三维偏序实现 Polylog。

* 例题 J

*J - 八云蓝自动机 I

给定长度为 n 的整数数组 A （初始值给定），以及长度为 m 的操作序列，每个操作是以下三种之一：

- 赋值： $A_x \leftarrow k$;
- 交换：交换 A_x 与 A_y ;
- 查询：输出当前 A_x 的值。

有 q 次询问，每次给出区间 $[l, r]$ 。对于每次询问，从初始数组 A 出发，依次执行操作 $l, l+1, \dots, r$ ，将所有查询操作输出的值累加，输出该累加和。

数据规模： $n, m, q \leq 2 \times 10^5$ 。

* 例题 J 题解

莫队。还是考虑删除与加入。注意到末尾加入一个和开头加入一个似乎差距很大，所以我们要分开考虑。（实现的时候也就不能用前面说到的那种技巧了）。

考虑将第一类问题转化为第二类问题。具体地，我们新建若干点 a_{n+r} ，值为赋值操作的 k ，则赋值就能转化为交换 a_x 与新建的 a_{n+r} 。

我们先来探索需要维护什么。假设是后面加入一个点，如果是查询操作，则需要知道现在的 a_x 是什么。假设是前面加入一个点，查询需要知道原来的 a_x ；如果是交换，则需要后面的计算会完全不同，需要知道多算了什么，且要知道一通交换之后，这个加入的交换实际最后导致了什么效果。

* 例题 J 题解

维护 b_x 表示交换完的 a 数组, a_x 表示初始状态, 同时维护 pos_x 表示一开始下标为 x 的点现在的下标, 同时维护 rev_x 为现在下标为 x 的点原来的下标。

为了处理“多算了什么”, 我们还要知道 t_x 表示现在在 x 的点计算了多少次。

具体地, 先考虑在后面加入一个点, 也就是从 $[l, r] \rightarrow [l, r + 1]$, 分两类:

- 如果是查询操作, 答案会加入 b_x , t_x 加一。
- 如果是交换操作, 则 b_x 和 b_y , rev_x 和 rev_y , pos_{rev_x} 和 pos_{rev_y} 交换。

* 例题 J 题解

如果是在前面加入一个点，即 $[l, r] \rightarrow [l-1, r]$ ，则：

- 如果是查询操作，则答案加入 a_x 。
- 如果是交换操作，则效果是使得原来在 x, y 的交换，即交换 a_{pos_x} 和 a_{pos_y} ， pos_x 和 pos_y ， rev_{pos_x} 和 rev_{pos_y} 。

之后，我们考虑这样会改变多少答案。显然要减去原来在 pos_x/pos_y 的位置的值乘上次数，加上现在的。我们用交换以后的数据，则减去

$a_{pos_x} \times t_{pos_y} + a_{pos_y} \times t_{pos_x}$ ，加上 $a_{pos_x} \times t_{pos_x} + a_{pos_y} \times t_{pos_y}$ 。也就是题解中的

$$(a_{pos_x} - a_{pos_y})(t_{pos_x} - t_{pos_y})$$

删除点是对称的，总的时间复杂度 $\mathcal{O}(n\sqrt{n})$ 。

回滚莫队

我们说到，莫队的关键是如何快速加入/删除点来扩展/缩小区间，但并不是所有东西都能够快速删除（比如最值问题）。所以，我们希望能有一种方法解决这样的问题。这就是回滚莫队：

回滚莫队

将询问按照正常莫队的顺序排。在处理左端点在一个块内的所有询问时，先将左端点置于下一个块的开头，右端点为 $l-1$ 表示目前为空。

每次询问时，先将左端点退回到下一个块的开头，然后移动左右端点。这样没有删除操作而只有加入操作。

关键在于如何“退回到下一个块的开头”。

回滚莫队

有两种方法，一种方法是保存左端点在下一个块开头时的信息，稍微控制地暴力恢复。另一种方法是注意到虽然我们删除，但是可以撤销。

具体地，从第一个询问开始，先移动右端点，然后将左端点从“下一个块开头”移动。这样下一个询问时，直接反过来做所有你做过的操作（第二种方法），或者用记录下来的修改过的地方恢复（第一种方法），将左端点退回去，再移动右端点、左端点即可。关键是要先移动右端点。

关于撤销，猜你想搜：~~Command 设计模式。（完全没必要。）~~

注意到回滚莫队不能处理左右端点在一个块内的答案，需要暴力处理。这样复杂度不退化，依然是 $O(n\sqrt{n})$ 。

Hint

反过来，如果一个东西容易删除不容易加入，也可以反着使用回滚莫队。

例题 K

K - 【模板】回滚莫队

给定一个序列，多次询问一段区间 $[l, r]$ ，求区间中相同的数的最远间隔距离。

$$1 \leq n, m \leq 2 \cdot 10^5$$

例题 K

K - 【模板】回滚莫队

给定一个序列，多次询问一段区间 $[l, r]$ ，求区间中相同的数的最远间隔距离。

$$1 \leq n, m \leq 2 \cdot 10^5$$

模板题。

目录

① 莫队

② 操作分块/根号重构

③ 杂题

操作分块/根号重构

操作分块是另一种技巧。一般用于各种数据结构题的非正解。

我们通常会遇到这样的题：给你一个东西，然后若干操作（修改/询问）。

操作分块的思想是，我们对这些操作进行分块，一个块结束时，将块内的修改操作一起执行，得到假设之后不再有修改时，下一个块内询问的答案，或者方便求出答案。然后对于下一个块内实际的询问，暴力看这个块内的每个操作对每个询问的影响，设块长 B 。复杂度 $\mathcal{O}(B^2)$ 。

听起来很抽象，我们来看例题。

例题 L

L - Xenia and Tree

给定一棵 n 个节点的树，初始时节点 1 为红色，其余节点为蓝色。

支持两种操作：

1. 将节点 v 染成红色；
2. 查询节点 v 到最近红色节点的距离。

数据规模： $n, m \leq 10^5$ 。

例题 L 题解

本题正解是动态点分治，但注意到只对树上单点进行操作并且操作简单，不妨考虑操作分块。

首先我们考虑如何在块中 $\mathcal{O}(B^2)$ 枚举时快速计算一个操作对一个询问的贡献，很明显是直接计算询问点与变红点在树上的距离然后取 $\min$ 。

然后考虑在本块之前就已经是红色的点，怎么去计算他们的贡献。我们发现我们可以接受 $\mathcal{O}(n)$ 级别的复杂度，所以可以直接多源 BFS，求每个点到最近红色点的距离。

“多源 BFS”

将多个起点全部塞入队列中，一个点被访问后不再入队。可以得到一个点距离若干起点的最短路径中的最短长度。

取 $B = \sqrt{n}$ 。总的复杂度 $\mathcal{O}(n\sqrt{n})$ 。

目录

① 莫队

② 操作分块/根号重构

③ 杂题

例题 M

M - [Violet] 蒲公英

给定长度为 n 的数组 a ，强制在线回答 m 个区间众数查询（若有多个，输出值最小的）。

$n \leq 4 \times 10^4$, $m \leq 5 \times 10^4$ 。

例题 M 题解

区间众数显然不能 Polylog，考虑分块。注意到即使针对固定查询合并两个块也是困难的，考虑使用类似例题 B 的做法。

设 $f_{l,r}$ 表示第 l 个块到第 r 个块内所有数的众数，同时维护 $g_{x,i}$ 表示 x 在前 i 个块内的出现次数。

最后的答案一定出自区间整块的 f 或者散块中出现的数，所以利用 g 数组和暴力统计散块内的出现次数计算众数即可。

时间复杂度 $\mathcal{O}((n+m)\sqrt{n})$ 。

例题 N

N - 秃子酋长

给一个长为 n 的排列 $a_1, \dots, a_n$, 有 m 次询问, 每次询问区间 $[l, r]$ 内, 排序后相邻的数在原序列中的位置的差的绝对值之和。

$$1 \leq n, m \leq 5 \times 10^5$$

例题 N 题解

考虑莫队。不难想到加入元素就是让答案加上这个元素与前驱和后继的位置差，同时减去被拆开的旧相邻对，删除元素就是反过来。需要一个 `set` 来查前驱后继。时间复杂度 $\mathcal{O}(n\sqrt{n}\log n)$ 。

注意到删除元素其实可以优化，我们类似链表地，维护每个点的前驱和后继具体是谁。只需要在插入的时候修改前后的信息即可。时间复杂度没有变化。

注意到删除元素现在变成了 $\mathcal{O}(1)$ ，于是我们考虑回滚莫队（的反向操作），不加入莫队。具体地，排序的第二关键字变为右端点从后到前，每次左端点从当前块的最左边开始向右移动即可。总的复杂度 $\mathcal{O}(n\sqrt{n})$ 。

例题 O

O - 桥梁

给定一张 n 点 m 边的无向图，第 i 条边带权 w_i ，有 q 次如下两种操作：

- $w_x \leftarrow y$
- 查询从 x 开始，只通过 $\geq y$ 的边，可达点数量。

$1 \leq n \leq 5 \times 10^4$, $0 \leq m \leq 10^5$, $1 \leq q \leq 10^5$, $w_i, y \leq 10^9$

例题 ○ 题解

若没有修改操作，可以使用 Kruskal 重构树，也可以离线，将询问按 y 从大到小排序，并查集维护即可。

有修改，不妨考虑操作分块，将 B 次操作划为一块。Kruskal 重构树的形态受修改操作影响，难以动态维护，于是我们考虑并查集。

对每一条边，维护出它在本块之前（前一块操作结束时）的最后一次修改，首先肯定是以边权序对询问排序，那么对块内无修改的边，它最后一次修改在当前块前，直接随边权序加入并查集即可。

例题 ○ 题解

再考虑块内有修改（二类修改）的边：仅有 B 条，考虑可撤销并查集，对于询问 (t, x, y) ，加入满足以下条件的边 (t_0, x_0, y_0) ：

- $y_0 \geq y$;
- 是满足 $t_0 \leq t$ 的、对 x_0 的最后一次修改。

算出答案后再撤销即可，这部分单次询问是 $\mathcal{O}(B)$ 的。

总复杂度 $\mathcal{O}((m \log m + B \log B) \cdot \frac{q}{B} + qB \log n)$ ，两个 $\log$ 分别来自排序和可撤销并查集，视 n, m, q 同阶，取 $B = q^{0.5}$ 时，有最优复杂度 $\mathcal{O}(q \log q \sqrt{q})$ 。

可撤销并查集

考虑不进行路径压缩，则每次合并时只会修改一个点的 fa 值。容易撤销。为了保证复杂度，我们使用按秩合并：合并前深度大的作为根，保证总的深度比较小。

* 例题 P

P - 众数

给定长度为 n 的整数序列 a ，允许选择任意区间 $[l, r]$ 和任意整数 k ，将区间内所有数加上 k （操作恰好一次）。

求操作后序列众数的最大出现次数，并输出所有能达到该最大次数的众数取值。

$\sum n \leq 5 \times 10^5$ ，3s。可以扩到 6s。

* 例题 P 题解

注意到 k 是任选的，所以答案一定可以取到 $[l, r]$ 内的众数 + 区间外的众数。问题转化为：任取 $[l, r]$ 最大化区间内的众数出现次数加区间外众数的出现次数。

考虑根号分治，阈值 B ，分三类讨论：

- 区间外众数出现次数大于 B 。
- 区间内众数出现次数大于 B 。
- 都小于 B 。

第一类，枚举区间外众数 x ，然后枚举区间内众数 y 。 $x = y$ 易，设 $x \neq y$ 。要最大化内外众数次数之和。设初始 $[l, r]$ 为空。若把一个 x 框进 $[l, r]$ ，会损失 1；一个 y 框进去，会得到 1。问题转化为令 $a_i = x$ 的位置为 -1 ， $= y$ 的为 1 ，求最大子段和。注意到， $a_l = a_r = y$ 一定不劣，所以我们可以 $\mathcal{O}(\text{cnt}_y)$ 的时间内解决。于是对于一个 x ，复杂度 $\mathcal{O}(n)$ ，总的复杂度 $\mathcal{O}(\frac{n^2}{B})$ 。

* 例题 P 题解

对于第二类（区间内众数出现次数大于 B ），枚举区间内众数 x ，然后枚举区间外众数 y 。类似上面的，我们令 x 为 1， y 为 -1 ，求最大子段和。当然有 $a_l = a_r = x$ 不劣，但是 cnt_x 比较大。

不过反过来卡，注意到 $a_{l-1} = y$ （或者 $l = 1$ ）且 $a_{r+1} = y$ （或者 $r = n$ ）一定不劣。这时复杂度为 $\mathcal{O}(cnt_y)$ ，不退化。

对于第三类，我们枚举区间外众数 x ，但是不能再枚举区间内众数了。不过一样分析，一定有 $a_{l-1} = x$ （或者 $l = 1$ ）且 $a_{r+1} = x$ （或者 $r = n$ ）一定不劣。

* 例题 P 题解

注意到一个小性质：

性质

满足条件的区间总共不超过 nB 个。

不考虑 $l = 1$ or $r = n$ 的情况。满足条件的区间一定有 $a_{r+1} = a_{l-1}$ 。又 $cnt_x < B$ ，所以对于一个 r 一定只有 $\mathcal{O}(B)$ 个 l 满足条件。□

所以第三类的做法保证了复杂度。

但是有问题，第三类由于没有枚举 y 所以我们不知道区间内众数出现次数是多少（以及是谁）。一个最无脑的做法就是直接莫队，通过调整块长可以做到 $\mathcal{O}(n^{1.75})$ 。估计没法卡过去，反正我 VP 这场比赛时卡了很久没过。

* 例题 P 题解

考虑从小到大枚举 r ，维护 S_i 表示 $[i, r]$ 的众数出现次数。当向右移动节点时，这样修改 S_i ：对于每个 $a_k = a_r$ ，设 $a_{k \sim r}$ 内有 p 个 a_r ，则 $S_{1 \sim k}$ 对 p 取 $\max$ 。

设每个值出现的次数为 t_i ，这样做的复杂度是平方和，即 $\sum_{i=1}^k t_i^2$ 。由于我们只需要考虑出现次数不超过 B 的 a ，所以有 $t_i \leq B$ ，此外 $\sum t_i \leq n$ ，所以由简单的放缩有：

$$\sum_{i=1}^k t_i^2 = \sum_{i=1}^k t_i \cdot t_i \leq B \cdot \sum_{i=1}^k t_i \leq B \cdot n = \mathcal{O}(nB)$$

取 $B = \sqrt{n}$ ，总的复杂度 $\mathcal{O}(n\sqrt{n})$ 。

常数比较玄学，所以开到了 6s。

另外，题解中说要“每次从 k 开始从右往左用 p 更新 S_i ，直到 $S_i \geq p$ 时退出”，并将此作为复杂度正确的保障，其实不用。但它可以作为卡常的技巧。